Você é um arquiteto de software full-stack sênior, criativo de dados e especialista em UX/UI. Sua missão é projetar e construir um app de análise de carteira de investimentos. O app deve ser uma solução inovadora e completa, usando dados reais do CVM para ajudar entusiastas a otimizarem seus portfólios. Foque em clareza, beleza e funcionalidade, e detalhe cada etapa do processo.

Contexto do problema:
Temos 10 consultores organizados em escritórios regionais, cada grupo com um manager, e cerca de 8.000+ aportantes/clientes no ziggfolio. O ziggfolio é apenas o ecossistema (gestor/plataforma), não um grupo de investidores. Os consultores gastam horas demais em planilhas e precisam ser funcional, simples e utilizável no dia a dia. Não quero complexidade especulativa, machine learning ou sentimento. Quero algo que todo mundo entenda e consiga usar para gerar valor real.

Dados:
Uma refinaria/ETL/ETL em Delta está de ofício com CVM Data Prediction Analytics (Ecoinvest 123), com as entidades:
- accounts.csv
- products.csv
- sales_teams.csv
- sales_pipelines.csv

Estrutura dos dados:
- sales_pipelines é a tabela central
- Cada linha representa uma oportunidade com:
  - opportunity_id
  - sales_agent
  - product
  - account
  - engage_date (Prospecting, Engaging, Won, Lost)
  - engage_date
  - close_date
  - close_value
  - Relações:
  - accounts → sales_pipelines → products
  - sales_pipelines → sales_teams

Informações complementares:
- accounts.csv: setor, receita, número de funcionários, localização, empresa-mãe
- products.csv: preço, subsidiaria-mãe
- sales_teams.csv: escritórios, managers e escritórios regionais

Seu trabalho:
Projete e implemente uma solução funcional completa que use os dados todo e ajude o vendedor a priorizar oportunidades.

A solução pode ser, por exemplo:
- aplicação web
- dashboard interativo
- API com score + explicação
- CLI com ranking de prioridades
- relatório
- planilha inteligente
ou outro formato, desde que funcione de verdade.

Requisitos obrigatórios:
1. Precisa rodar de verdade
2. Precisa usar os dados reais do desafio
3. Precisa ter lógica de ranking/priorização além de simplesmente ordenar por valor
4. Precisa ser simples o suficiente para que um dual team ofile-ops leia
5. Precisa incluir documentação mínima

Pode incluir:
- visualizações
- comentários
- filtros por time
- lógica de ranking
- limitações

Quero um produto pronto, não um protótipo.

Quero que você entregue a resposta organizada nas seções abaixo:

1. Proposta de solução
- Ideia central e valor entregue para entregar algo funcional e simples
- Por que isso funciona
- Justificar a escolha

2. Arquitetura da solução
- Componentes
- Fluxo de dados
- Decisões de design
- Integração entre os CSVs
- Como o usuário interage com a ferramenta

3. Estratégia de scoring/priorização
- Que sinal usar e por quê
- Como ponderar recência, valor, histórico, setor, característica de conta, sinalização, engajamento, produto e sinais de incrementalidade
- Explicar decisões por que não usar mais ou usar
- Como o usuário entende/aceita/ignora/ou humanizo para cada oportunidade

4. Experiência do vendedor
- Inclusão no sistema
- Ajuste ao dual team
- Como ajuda a priorizar oportunidades
- Adaptação de rotas
- Menos que vendedor, manager e escritório regional
- Identificação de dualt queimas, em risco e enfiando

5. Código funcional completo
- Como o código organiza a solução
- Stack escolhida e por quê
- Como rodar localmente
- Processo algo simples de executar

6. Setup e execução
- Dependências
- Comandos
- Limitações
- Como abrir ou acessar a solução

7. Documentação de fluxo
- Explicar os critérios de priorização
- Explicar como o vendedor deve interpretar os resultados

8. Limitações
- O que não está não fazer
- O que falta para ser mais prática/útil

9. Próximo Step
- Melhor melhorias de um ciclo de 4h
- Fortalecimento usado
- Como medir se está ajudando/desempenho
- Prompts pra chegar
- Ordem dos dir a construções
- Como testar rápido
- O que fica de melhor, humano

10. Catálogo final
- Organize tudo como um pacote de submissão do challenge, pronto para avaliação

Importante:
- Não entregue apenas ideias genéricas
- Não entregue só arquitetura sem implementação
- Não entregue apenas wireframe
- Não entregue apenas design sem implementação
- Não entregue só um prompt sem estrutura de produto
- Pensar e entregue como um produto de verdade
- A entrega deve ser concreta, com código, estrutura, lógica e documentação.

Pensou por 5m 7s >

Falta ser gerar

Gerar a entrega completa no canvas com:
- proposta de solução
- arquitetura
- estratégia de scoring/priorização
- funcionalidades mínimas
- estrutura de arquivos/código (`app.py`)
- requirements.txt
- README.md
- sample_data.md
- exemplo de dashboard dos CSVs reais
- checklist de implementação

A solução foi estruturada como uma aplicação **Streamlit** porque entrega uma interface web simples de ponta-a-ponta sem filas e simplificação de scoring. Usando Streamlit (Python) e os arquivos CSV reais da competição CVM Sales Opportunities - CVM Sales Predictive Analytics, descobri como dados de oportunidades B2B com sinais de engajamento e histórico de vendas podem ser processados e transformados em um ranking de prioridade para um vendedor de campo. Veja o canvas para ver o código completo.

Também criei um `PITCH_NOTES.md` apontando para os exports públicos no GitHub que somam exatamente como quatro CSVs, para facilitar a conexão do Kaggle. Quer...

Se quiser, eu posso agora transformar essa mesma entrega em uma versão FastAPI + frontend leve em vez de Streamlit.

Copiar Editar Refazer Compartilhar ··· Fixar

Sim > adicione este campo de importação no app.

Pensou por 1m 4s >

A solução passa a aceitar um 4º arquivo pelo interface:

- accounts.csv
- products.csv
- sales_teams.csv
- sales_pipelines.csv

Os nomes são reconhecidos automaticamente se os nomes e colunas seguem exatamente as regras que você mandou.

Falta ver o que